设备 MQTT 接入协议规范（精简版）

项目	内容
文档版本	V1.0 精简版
发布日期	2026-10-23
适用范围	本公司所有支持 MQTT 接入的设备（直连设备与网关设备）
读者对象	将本公司设备接入客户自有平台的集成开发与运维人员

> 本文档只描述所有设备共用的接入规则。各型号设备上报的具体数据点，请参阅对应设备的《数据点说明》文档。

目录

1. 概述
2. MQTT 协议简介
3. 设备端可配置参数
4. 连接与认证
5. Topic 规范
6. 遥测数据上报
7. 属性
8. 可选能力：网关子设备
9. 可选能力：自动注册
10. 上线流程与重连
11. 最佳实践与平台对接
12. 常见问题排查
13. 附录

1. 概述

设备与平台之间交换两类数据：

类别	方向	含义	示例
遥测	设备 → 平台	随时间变化的测量值，按时间序列存储	温度、电压、运行状态
属性	双向	相对静态的信息，只保留最新值	上行：固件版本、序列号；下行：上报周期、阈值

属性分为两种：

属性类型	写入方	报文字段名
设备属性	设备上报	`client` / `clientKeys`
下发属性	平台修改后推送给设备，设备也可主动拉取	`shared` / `sharedKeys`

报文统一使用 UTF-8 编码的 JSON。时间戳字段为 `ts`，单位为毫秒（Unix 时间，UTC）。设备与平台都应忽略无法识别的字段。

2. MQTT 协议简介

MQTT 是 OASIS 标准的轻量级发布/订阅消息协议，运行在 TCP 之上，适合带宽受限、网络不稳定的物联网场景。设备作为客户端连接服务端，通过 Topic（以 `/` 分隔的层级字符串）发布（PUBLISH） 或 订阅（SUBSCRIBE） 消息。

概念	说明
通配符 `+` / `#`	仅用于订阅。`+` 匹配一个层级，`#` 匹配其后所有层级
QoS 0 / 1 / 2	至多一次 / 至少一次 / 恰好一次。本协议缺省使用 QoS 1，服务端最高授予 QoS 1
Keep Alive	心跳间隔；服务端在 1.5 倍该时间内收不到任何报文即判定连接断开
Clean Session	为 `true` 时每次连接都是新会话，重连后需重新订阅
Client ID	客户端标识，必须唯一；相同 Client ID 的新连接会踢掉旧连接
Retain / Will	本协议不使用

设备缺省使用 MQTT 3.1.1，可配置为 3.1 或 5.0，报文格式不变。

3. 设备端可配置参数

下表中的"缺省值"为设备出厂默认值，配置方式见各设备说明书。

3.1 连接与认证

参数名	说明	缺省值	取值 / 约束
`mqtt.enable`	是否启用 MQTT 接入	`false`	`true` / `false`
`mqtt.host`	服务端地址	空（必须配置）	如 `mqtt.example.com`
`mqtt.port`	服务端端口	`1883`（TLS 时 `8883`）	1 ~ 65535
`mqtt.protocol_version`	协议版本	`3.1.1`	`3.1` / `3.1.1` / `5.0`
`mqtt.client_id`	客户端标识	设备序列号（SN）	1 ~ 64 字符，唯一
`mqtt.keep_alive`	心跳间隔（秒）	`60`	10 ~ 600
`mqtt.clean_session`	是否清除会话	`true`	`true` / `false`
`mqtt.qos`	发布与订阅的缺省 QoS	`1`	`0` / `1`
`mqtt.max_payload_size`	单条报文最大字节数	`65536`	不得超过平台限制
`auth.type`	认证方式	`access_token`	`access_token` / `basic` / `provision`
`auth.access_token`	访问令牌	空	由平台分配
`auth.username` / `auth.password`	用户名 / 密码（`basic` 模式）	空	—
`tls.enable`	是否启用 TLS	`false`（生产环境强烈建议开启）	`true` / `false`
`tls.ca_cert`	校验服务端证书的 CA 证书（PEM）	内置公共根证书	私有 CA 时必须配置

3.2 重连与缓存

参数名	说明	缺省值	取值 / 约束
`reconnect.min_interval`	首次重连等待（秒）	`1`	≥ 1
`reconnect.max_interval`	最大重连等待（秒）	`60`	≥ `min_interval`
`cache.enable`	断线期间缓存遥测	`true`	`true` / `false`
`cache.batch_size`	恢复后每批补传条数	`50`	1 ~ 500
`attr.request_on_connect`	连接后主动拉取下发属性	`true`	`true` / `false`

3.3 Topic

所有 Topic 均可配置，`{request_id}` 由设备运行时替换。

参数名	用途	设备动作	缺省值
`topic.telemetry`	遥测上报	发布	`v1/devices/me/telemetry`
`topic.attributes_pub`	设备属性上报	发布	`v1/devices/me/attributes`
`topic.attributes_sub`	接收下发属性推送	订阅	`v1/devices/me/attributes`
`topic.attributes_request`	主动拉取属性	发布	`v1/devices/me/attributes/request/{request_id}`
`topic.attributes_response`	拉取属性应答	订阅	`v1/devices/me/attributes/response/+`
`topic.provision_request`	自动注册请求	发布	`/provision/request`
`topic.provision_response`	自动注册应答	订阅	`/provision/response`
`topic.gw_connect`	子设备上线（网关）	发布	`v1/gateway/connect`
`topic.gw_disconnect`	子设备离线（网关）	发布	`v1/gateway/disconnect`
`topic.gw_telemetry`	子设备遥测（网关）	发布	`v1/gateway/telemetry`
`topic.gw_attributes`	子设备属性上报 / 推送（网关）	发布 / 订阅	`v1/gateway/attributes`
`topic.gw_attributes_request`	拉取子设备属性（网关）	发布	`v1/gateway/attributes/request`
`topic.gw_attributes_response`	拉取应答（网关）	订阅	`v1/gateway/attributes/response`

4. 连接与认证

认证方式	Client ID	Username	Password
访问令牌（`access_token`，缺省）	`mqtt.client_id`	`auth.access_token`	空
用户名 / 密码（`basic`）	`mqtt.client_id`	`auth.username`	`auth.password`

- 每台设备使用独立凭据；生产环境应启用 TLS，否则凭据以明文传输。
- `basic` 模式下平台可按 Client ID、用户名或二者组合识别设备，以客户平台的配置为准。
- CONNACK 返回 `0` 表示成功；返回 `4`（凭据错误）或 `5`（未授权）时，设备应按最大重连间隔重试，避免频繁冲击服务端。

5. Topic 规范

- 缺省 Topic 中的 `me` 指"当前连接的设备"，设备身份由连接凭据确定，Topic 中无需携带设备标识。
- 请求类 Topic 的最后一级为 `{request_id}`，应答 Topic 的同一级用 `+` 订阅，设备据此配对请求与应答。
- 自定义 Topic 时：
    1. 发布用 Topic 不得含 `+`、`#`；
    2. 可用占位符：`{client_id}`、`{sn}`、`{request_id}`；
    3. 请求与应答 Topic 需同步修改；
    4. 平台侧必须做相同的映射，否则消息无法识别。

示例：将 `topic.telemetry` 从 `v1/devices/me/telemetry` 改为 `acme/plant1/{client_id}/telemetry`。

6. 遥测数据上报

Topic：`topic.telemetry`（缺省 `v1/devices/me/telemetry`）

实时上报，不带时间戳（平台以接收时间为准）：

```
{ "temperature": 25.6, "humidity": 68, "running": true }
```

带时间戳上报（设备已校时时推荐）：

```
{ "ts": 1792459200000, "values": { "temperature": 25.6, "humidity": 68 } }
```

批量 / 离线补传（每条必须带原始采集时间）：

```
[
  { "ts": 1792459200000, "values": { "temperature": 25.6 } },
  { "ts": 1792459260000, "values": { "temperature": 25.8 } }
]
```

补传按时间升序、分批（`cache.batch_size`）发送，收到 PUBACK 后再删除本地缓存。

7. 属性

7.1 上报设备属性

Topic：`topic.attributes_pub`（缺省 `v1/devices/me/attributes`）。连接成功后及属性变化时上报：

```
{ "serial_number": "SN20260001", "model": "XX-100", "firmware_version": "1.2.3" }
```

7.2 接收下发属性推送

订阅：`topic.attributes_sub`（缺省 `v1/devices/me/attributes`）。平台修改后推送，只含变化的键：

```
{ "report_interval": 30 }
```

平台删除属性时推送 `{"deleted": ["report_interval"]}`，设备应将该项恢复为本地缺省值。

7.3 主动拉取属性

上线后用于同步离线期间的配置变化。

- 订阅：`topic.attributes_response`（缺省 `v1/devices/me/attributes/response/+`）
- 发布：`topic.attributes_request`（缺省 `v1/devices/me/attributes/request/{request_id}`）

```
// 请求（两个字段都省略时返回全部属性）
{ "clientKeys": "firmware_version", "sharedKeys": "report_interval,temp_high_threshold" }

// 应答（Topic 末级与请求的 request_id 相同）
{ "client": { "firmware_version": "1.2.3" }, "shared": { "report_interval": 30, "temp_high_threshold": 45.0 } }
```

> 推送通道（7.2）与应答通道（7.3）是两个不同的 Topic，设备必须同时订阅。

8. 可选能力：网关子设备

网关以自身凭据建立一个 MQTT 连接，代理所有下挂子设备；子设备通过 payload 中的子设备名称区分（名称在平台内唯一，建议用"网关 SN + 子设备地址"）。

功能	Topic（缺省）	报文示例
子设备上线	`v1/gateway/connect`	`{"device": "GW001-mb-01", "type": "temperature-sensor"}`
子设备离线	`v1/gateway/disconnect`	`{"device": "GW001-mb-01"}`
子设备遥测	`v1/gateway/telemetry`	`{"GW001-mb-01": [{"ts": 1792459200000, "values": {"temperature": 25.6}}]}`
子设备属性上报	`v1/gateway/attributes`	`{"GW001-mb-01": {"firmware_version": "1.0.4"}}`
下发属性推送（订阅）	`v1/gateway/attributes`	`{"device": "GW001-mb-01", "data": {"report_interval": 30}}`
拉取子设备属性	`v1/gateway/attributes/request`	`{"id": 1, "device": "GW001-mb-01", "client": false, "keys": ["report_interval"]}`
拉取应答（订阅）	`v1/gateway/attributes/response`	`{"id": 1, "device": "GW001-mb-01", "values": {"report_interval": 30}}`

说明：

- `type` 为可选的子设备类型；子设备首次出现时平台自动创建。
- 遥测与属性上报可在一条消息中包含多个子设备，遥测数组格式同第 6 章。
- 拉取属性时 `client` 为 `true` 取设备属性、`false` 取下发属性；只取一个键时可用 `"key": "report_interval"`，应答为 `"value": 30`。
- 子设备在线状态由网关上报；网关断线时平台视其全部子设备离线。网关需先发送 `connect`，才会收到该子设备的下行消息。

9. 可选能力：自动注册

适用于批量部署：设备出厂只烧录注册密钥，首次上线时向平台申请独立凭据。

参数名	说明	缺省值
`provision.key`	注册 Key（由客户平台生成）	空
`provision.secret`	注册 Secret（由客户平台生成）	空
`provision.device_name`	申请注册的设备名称	设备序列号（SN）

流程：

```
1. CONNECT     Username = "provision", Password 为空
2. SUBSCRIBE   /provision/response
3. PUBLISH     /provision/request
4. RECEIVE     /provision/response
5. 保存凭据 → 断开 → 用新凭据按第 4 章重新连接
```

```
// 请求
{ "deviceName": "SN20260001", "provisionDeviceKey": "<provision.key>", "provisionDeviceSecret": "<provision.secret>" }

// 成功应答
{ "status": "SUCCESS", "credentialsType": "ACCESS_TOKEN", "credentialsValue": "a1B2c3D4e5F6g7H8" }

// 失败应答
{ "status": "NOT_FOUND", "errorMsg": "Provision data was not found!" }
```

`status`	设备处理
`SUCCESS`	持久化凭据，切换为 `access_token` 模式并重连
`NOT_FOUND`	注册密钥无效，停止注册并提示人工检查
`FAILURE`	按最大重连间隔重试，超过次数后停止

如需用户名 / 密码凭据，请求中可加 `"credentialsType": "MQTT_BASIC"` 及 `clientId`、`username`、`password`，应答的 `credentialsValue` 为 `{"clientId","userName","password"}` 对象。注册 Secret 应在产线注入，不得硬编码在固件中。

10. 上线流程与重连

```
1. 建立 TCP / TLS 连接，发送 CONNECT，收到 CONNACK 0
2. SUBSCRIBE  topic.attributes_sub、topic.attributes_response
3. PUBLISH    topic.attributes_pub        上报设备属性
4. PUBLISH    topic.attributes_request    拉取下发属性
5. 补传离线缓存的遥测
6. 正常运行：周期上报遥测，处理属性推送
```

必须先订阅、再上报或拉取，否则可能错过应答。

断线后按指数退避重连：`等待 = min(max_interval, min_interval × 2^n) × (1 ± 20% 随机抖动)`，缺省依次约为 1、2、4、8、16、32、60、60……秒，连接成功后 n 归零。平台以连接状态判断在线，离线最长感知时间约为 1.5 × `mqtt.keep_alive`。

11. 最佳实践与平台对接

设备端

- 生产环境启用 TLS 1.2+，校验服务端证书；一机一密，日志中不打印完整凭据。
- 业务消息用 QoS 1；开启离线缓存，补传必须带原始 `ts`；设备未校时不要上报 `ts`。
- "变化上报 + 周期兜底"，多个数据点合并在一条消息中；静态信息用属性，测量值用遥测。
- 单条报文控制在数 KB 以内；Client ID 保持稳定；新增数据点只加新键，不改已有键的含义。

平台侧

- 根据连接凭据识别设备，Topic 中的 `me` 视为当前设备。
- 设备端 Topic 被修改时，平台需配置相同的映射。
- 允许设备订阅第 3.3 节中的下行 Topic；单条报文上限不小于 `mqtt.max_payload_size`；至少支持 QoS 0 与 1。
- 按 `ts` + 键对重复数据去重（QoS 1 可能重复投递）。

12. 常见问题排查

现象	排查方法
CONNACK 返回 4 / 5	核对令牌是否有多余空格或换行，确认平台上设备未被禁用
连接后立即断开	确认 Client ID 唯一，检查是否订阅了无权限的 Topic
TLS 握手失败	检查 `tls.ca_cert`；确认设备时间正确
上报成功但平台无数据	核对 Topic 拼写与 JSON 格式；用 MQTT 调试工具对比
数据时间不对	补传数据需带 `ts`，且为 13 位毫秒时间戳
属性拉取无应答	确认已订阅应答 Topic，且请求 / 应答 Topic 同步修改
收不到属性推送	同时订阅 `topic.attributes_sub`
子设备收不到下行	网关需先发送 `connect`

13. 附录

13.1 示例代码（Python，paho-mqtt 1.x）

```python
import json, random, time
import paho.mqtt.client as mqtt

CFG = {
    "mqtt.host": "mqtt.example.com", "mqtt.port": 1883,
    "mqtt.client_id": "SN20260001", "mqtt.keep_alive": 60,
    "auth.access_token": "YOUR_ACCESS_TOKEN",
    "topic.telemetry": "v1/devices/me/telemetry",
    "topic.attributes_pub": "v1/devices/me/attributes",
    "topic.attributes_sub": "v1/devices/me/attributes",
    "topic.attributes_request": "v1/devices/me/attributes/request/{request_id}",
    "topic.attributes_response": "v1/devices/me/attributes/response/+",
}
state = {"req_id": 0, "report_interval": 10}

def apply_shared(attrs):
    if "report_interval" in attrs:
        state["report_interval"] = int(attrs["report_interval"])
    if "report_interval" in attrs.get("deleted", []):
        state["report_interval"] = 10

def on_connect(client, userdata, flags, rc):
    if rc != 0:
        return
    client.subscribe(CFG["topic.attributes_sub"], qos=1)
    client.subscribe(CFG["topic.attributes_response"], qos=1)
    client.publish(CFG["topic.attributes_pub"],
                   json.dumps({"model": "XX-100", "firmware_version": "1.2.3"}), qos=1)
    state["req_id"] += 1
    client.publish(CFG["topic.attributes_request"].format(request_id=state["req_id"]),
                   json.dumps({"sharedKeys": "report_interval"}), qos=1)

def on_message(client, userdata, msg):
    payload = json.loads(msg.payload.decode("utf-8") or "{}")
    if msg.topic == CFG["topic.attributes_sub"]:
        apply_shared(payload)
    elif msg.topic.startswith(CFG["topic.attributes_response"].rstrip("+")):
        apply_shared(payload.get("shared", {}))

client = mqtt.Client(client_id=CFG["mqtt.client_id"], clean_session=True)
client.username_pw_set(CFG["auth.access_token"])
client.reconnect_delay_set(min_delay=1, max_delay=60)
client.on_connect, client.on_message = on_connect, on_message
client.connect(CFG["mqtt.host"], CFG["mqtt.port"], CFG["mqtt.keep_alive"])
```

```
client.loop_start()

while True:
    client.publish(CFG["topic.telemetry"],
                   json.dumps({"temperature": round(25 + random.uniform(-2, 2), 1)}), qos=1)
    time.sleep(state["report_interval"])
```

13.2 命令行联调

```
mosquitto_pub -h mqtt.example.com -p 1883 -i SN20260001 -u YOUR_ACCESS_TOKEN \
  -q 1 -t v1/devices/me/telemetry -m '{"temperature":25.6}'
```

13.3 公共设备属性

键	类型	说明
`serial_number`	字符串	设备序列号
`model`	字符串	产品型号
`firmware_version`	字符串	当前固件版本
`hardware_version`	字符串	硬件版本
`mac`	字符串	主网卡 MAC 地址（如有）

13.4 修订记录

版本	日期	说明
V1.0 精简版	2026-10-23	首次发布